

dS IBP Package 用户手册

Su Yingze^a

^a*Institute not specified*

E-mail: yingze.su@outlook.com

ABSTRACT: 本手册说明dS IBP package 的费曼规则、统一积分表示、massive/massless 指标约定、time 与loop-momentum IBP、独立变量微分方程seed、EOM/缩并canonical、拓扑输入、公开API、Kira 接口和tree vertex family。当前主线为016: 用户显式分别声明任意命名的loopExternalMomenta 与independentExternalMomenta, 程序验证结构圈数、routing、声明完备性和动力学坐标, 不自动猜角色。前者使用完整ssij 根号Gram 坐标, 后者只为用户声明的实际无圈模长生成sE1,sE2,...。package 只生成、序列化和验证关系, 不运行reduction。

Contents

1	快速开始与当前边界	2
2	第一个完整例子: bubble+tree	3
3	SK 费曼规则到积分族	6
4	统一三槽积分表示	7
5	Massive 导数、EOM 与缩并	8
6	Massless 有向单 n 与time-IBP	10
7	Topology、标量积、ISP 与外腿能量	12
8	IBP 生成元与公开接口状态	17
9	Seed、linearData 与serializer	18
10	016 标准package 与初始化	20
11	016 运行消息与进度	21
12	016 Kira 结果取回到微分方程	22
13	Tree vertex family	22
14	常见错误与当前限制	24
A	公开函数汇总	25

1 快速开始与当前边界

本package生成dS圈图的canonical IBP seed，并把关系转换为后端中立的linearData。当前稳定主线为模块化016，支持topology-driven的任意圈数、任意拓扑以及massive/massless混合内线；标准入口和冻结单文件兼容入口分别为

```
000_code/016_dSIBP/  
independent-benchmark/package/package_016.wl
```

010-015 是只读历史/基线版本。016 继承标准context、初始化metadata、完整time/loop-momentum生成元、massive EOM、massless有向单 n 、共同-theta odd-subset contact、contact可达sector、跨sector coincident canonical、tree迭代与dlog、Kira serializer/importer以及DE/scaling检查，并增加显式双动量列表、图论/routing审计、cycle/fixed line pack、direct pure-time路线、参数重定义和capability门禁。

仓库内标准加载与独立benchmark单文件加载是两个等价入口，应在新kernel中任选其一：

```
packageDir = FileNameJoin[{Directory[], "000_code", "016_dSIBP"}];  
If[!MemberQ[$Path, packageDir], AppendTo[$Path, packageDir]];  
Needs["dSIBP"];
```

(* 独立 benchmark 的自包含单文件入口: *)

```
Get["independent-benchmark/package/package_016.wl"];
```

两者都提供dSIBP'公开context；独立交付文件不依赖主线模块路径。不要在同一kernel重复加载两个入口。

推荐工作流为

1. 输入顶点、内线、圈/外动量基、ISP、零点和必要的数值规则；
2. 用DSInit验证输入、ISP坐标、函数系统与contact-reachable sectors；
3. 用DSSeeds生成每个sector的全部time与loop-momentum seed；需要完整离散态时显式传DiscreteMode->"all"；
4. seed层立即执行EOM/symmetry/canonical，并保留Mathematica表达式；
5. 用DSLinear构造后端中立linearData；必要的小规模数值规则只在这一层代入；
6. 按需调用DSKiraExport；用户在package外运行reduction；
7. 有完整外部结果时用DSKiraImport -> DSDE -> DSScaleCheck取回并验证。

解析seed只允许小规模生成。禁止默认展开整族解析IBP，也禁止为了验证做宽范围撒点。所有WolframScript检查必须显示消息运行，不能用Quiet掩盖问题。

给定下文的case后，最短的可执行调用链是

```

context = DSInit[case];
seeds = DSSeeds[
  context,
  DiscreteMode -> "all",
  GenerateShrinkSectors -> True
];
linearData = DSLinear[seeds, context];
{context["status"], seeds["dSIBPStatus"],
 linearData["dSIBPStatus"]}

```

每个批量接口都返回Association; 必须先检查"status", 再读取方程或传给下一阶段。

独立benchmark 的package/examples/ 提供六个应用例子:

```

06_root_kinematic_coordinates/main.wl
01_mixed_bubble_workflow.wl
02_function_system_hankel.wl
03_two_loop_isp.wl
04_pure_massive_bubble_closed_loop/main.wl
05_tree_two_vertex_time_ibp/main.wl

```

建议先运行第一行的bubble+tree 经典例子; 它展示016 的显式动量角色、ssij/sEe 初始化、参数重定义、总导数与错误输入门禁。其余例子依次展示mixed massive/massless 工作流、一般P,Q,T,W 函数系统、两圈ISP、pure massive bubble 的外部Kira 闭环, 以及两顶点tree pure-time/dlog 路线。examples 不含独立expected; Kira example 只写输入并取回已有结果, 不启动reduction。

2 第一个完整例子: bubble+tree

2.1 先按独立相位能量走通基准流程

考虑三个时间顶点。 v_1, v_2 之间的bubble 有两条cycle lines, 动量分别为 l_1 与 $l_1+k_1+k_2$; v_2, v_3 之间有一条无圈bridge line, 动量为 k_1+k_2 。 v_1 的另一侧外腿也携带 k_1+k_2 , v_3 有两条外腿 k_1, k_2 。输入中的加减法始终是矢量线性组合:

$$|k_1 + k_2| = \sqrt{\text{sp}(k_1 + k_2, k_1 + k_2)} \neq |k_1| + |k_2| \quad (2.1)$$

(一般运动学下)。 nu_1, nu_2, nu_3 是三条massive lines 的Hankel 指标; E_1, E_2, E_3 是三个顶点指数相位的独立标量。package 不从前两类对象推断第三类。

完整核心输入为

```

case = <|
  "name" -> "016BubbleTreeK1K2",
  "vertexData" -> {{v1, "+"}, {v2, "+"}, {v3, "+"}},
  "lineData" -> {
    <|"id"->1, "endpoints"->{v1, v2}, "momentum"->l1,
      "nu"->nu1, "bbType"->"h", "massType"->"massive"|>,

```

```

    <|"id"->2,"endpoints"->{v1,v2},"momentum"->l1+k1+k2,
      "nu"->nu2,"bbType"->"h","massType"->"massive"|>,
    <|"id"->3,"endpoints"->{v2,v3},"momentum"->k1+k2,
      "nu"->nu3,"bbType"->"h","massType"->"massive"|>
  },
  "extLegs" -> {
    {bubbleLeg,v1,k1+k2},{treeLeg1,v3,k1},{treeLeg2,v3,k2}
  },
  "loopMomenta" -> {l1},
  "loopExternalMomenta" -> {k1+k2},
  "independentExternalMomenta" -> {k1,k2},
  "ibpMode" -> "full",
  "vertexEnergies" -> <|v1->E1,v2->E2,v3->E3|>,
  "ispData" -> {},
  "zeroPointRules" -> {
    a0[v1]->alpha1,a0[v2]->alpha2,a0[v3]->alpha3,
    b0[1]->beta1,b0[2]->beta2,b0[3]->beta3
  },
  "symmetryRules" -> {},
  "seedPreset" -> "quickCheck"
|>;

```

先查看提案，再初始化：

```

proposal = DSKinematics[case];
proposal["selectionTemplate"]

context = DSInit[
  case,
  GenerateDerivativeMetadata -> True,
  ProgressReporting -> True
];
topology = context["topology"];
notation = DSPParameterNotation[context];

```

该输入的必要loop 方向只有 k_1+k_2 。它的完整一维Gram 已覆盖bridge 的模长； k_1,k_2 只补齐实际出现的两个独立无圈模长。缺省规则严格为

```

sp[k1+k2,k1+k2] -> ss11^2
sp[k1,k1]        -> sE1^2
sp[k2,k2]        -> sE2^2

```

因此 ss_{11} 是 $|k_1 + k_2|$ ，而 sE_1+sE_2 才是 $|k_1| + |k_2|$ 。短名必须与DSPParameterNotation 返回的原始sp binding 一起阅读； ss_{ij} 在 $i \neq j$ 时是Gram 点积坐标的根，不表示 $|k_i + k_j|$ 。

初始化后可以检查公开原子关系和批量 workflows：

```

integral = J[
  {0,0,0},
  {{0,0,0},{0,0,0},{0,0}},
  {}
];

timeSeed = dtau[v3,integral,topology];
loopLoopSeed = dqq[1,1,integral,topology];
loopExternalSeed = dqk[1,1,integral,topology];
totalDerivative = ds[ss11^2 integral+sE1 integral,sE1,topology];

seeds = DSSeeds[context,DiscreteMode->"all"];
linearData = DSLinear[seeds,context];

cycle line pack 为{b,n1,n2}, bridge pack 只有{n1,n2}; bridge 的动量幂进入显式ss11
系数, 不伪造b 指标。解析seed 不要求给数值规则; 若进入数值linearData/Kira 输入,
才需要补齐ss11,E1,E2,E3,nu1,nu2,nu3 等数值。

```

2.2 方案一: 保留两条外腿, 把模长和选作坐标

如果同一顶点的两个外腿指数需要合并成

$$E_0 = |k_1| + |k_2|, \quad (2.2)$$

是否采用 E_0 是用户的运动学参数选择, 不由package 猜测。保留原topology 时, 先把v3 的相位能量显式改为 E_0 , 再做满秩坐标重定义:

```

boundCase = Join[
  case,
  <|"vertexEnergies"-><|v1->E1,v2->E2,v3->E0|>|>
];
boundContext0 = DSInit[boundCase,GenerateDerivativeMetadata->True];

boundContext = DSRedefineParameters[boundContext0,{
  sp[k1+k2,k1+k2] -> ss11^2,
  sp[k1,k1]       -> (E0-sE2)^2,
  sp[k2,k2]       -> sE2^2
}];

```

数学上这正是 $sE1 \rightarrow E0 - sE2, sE2 \rightarrow sE2$, 但公开API 的规则左端必须写baseCoordinateOrder 中的原始sp[...], 不能直接写 $sE1 \rightarrow \dots$ 。该坐标图的可求导变量为

```
{ss11,E0,sE2,E1,E2}
```

其中 E_0 只出现一次; ds[expr,E0,boundContext] 同时对显式系数、模长binding 和v3 相位求导。平方规则选择了 $|k_1| = E_0 - sE2$ 的物理支, 使用者应在所研究区域保证 $E_0 > sE2 > 0$; 若需要别的分支, 必须显式更换参数化。

只改坐标而仍令 $v3 \rightarrow E3$, 不会建立 $E3 = E0$; 相反, 只令 $v3 \rightarrow E0$ 而不重定义两个模长, 也不会告诉package $E_0 = sE1 + sE2$ 。这两部分必须同时给出。

2.3 方案二：从输入起采用单一有效外腿

另一种做法是把v3 的两条外腿替换成一条有效外腿。此时topology metadata 已改变，不能把它当作上一个family 的纯坐标改名。为避免同一符号既作矢量又作标量，使用p0 标记有效外腿三动量，用E0 标记其模长和顶点相位：

```
singleLegCase = Join[case,<|
  "name" -> "016BubbleTreeSingleEffectiveLeg",
  "extLegs" -> {
    {bubbleLeg,v1,k1+k2},
    {treeLeg0,v3,p0}
  },
  "independentExternalMomenta" -> {p0},
  "vertexEnergies" -> <|v1->E1,v2->E2,v3->E0|>
|>];

singleLegContext0 = DSInit[
  singleLegCase,
  GenerateDerivativeMetadata->True
];
singleLegContext = DSRedefineParameters[singleLegContext0,{
  sp[k1+k2,k1+k2] -> ss11^2,
  sp[p0,p0] -> E0^2
}];
```

此时v3 只有一条外腿，独立变量为{ss11,E0,E1,E2}。package 只知道 $|p_0| = E_0$ ；“这个 E_0 对应原问题的 $|k_1| + |k_2|$ ”是用户在两个不同topology 之间建立的外部物理对应，不再能从新输入中恢复原来的两个独立模长，也不能分别对它们求导。若后续仍需要 k_1, k_2 的独立变化、交换对称性或软极限，应使用方案一而不是删除两条外腿。

3 SK 费曼规则到积分族

3.1 顶点规则

每个时间顶点 v 有共形时间 $\tau_v < 0$ 和SK 分支 $\sigma_v = \pm$ 。本package 使用

$$\sigma_v = + : \quad e^{-iE_v\tau_v}, \quad \sigma_v = - : \quad e^{+iE_v\tau_v}. \quad (3.1)$$

因此外腿相位对time-IBP 的贡献为

$$\partial_{\tau_v} e^{-iE_v\tau_v} = -iE_v e^{-iE_v\tau_v}, \quad \partial_{\tau_v} e^{+iE_v\tau_v} = +iE_v e^{+iE_v\tau_v}. \quad (3.2)$$

E_v 表示连接该顶点的所有外腿模长在指数中形成的打包能量。它不一定是进入圈内线动量的外部向量。

3.2 内线的四种类型

设内线 e 的有序端点为 $\{u_e, v_e\}$, 动量为

$$Q_e = \sum_{\ell=1}^L c_{e\ell} \mathbf{q}_\ell + \sum_{j=1}^K d_{ej} \mathbf{k}_j, \quad q_e = |Q_e|. \quad (3.3)$$

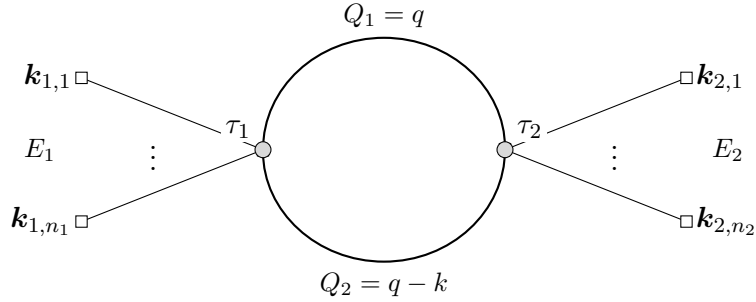
端点的SK 符号决定传播子类型:

质量类型	端点分支	指标包
massive	同分支++ / --	$\{\mathbf{b}[e], \mathbf{n}[e, 1], \mathbf{n}[e, 2]\}$
massive	异分支+- / -+	$\{\mathbf{b}[e], \mathbf{n}[e, 1], \mathbf{n}[e, 2]\}$
massless	同分支++ / --	$\{\mathbf{b}[e], \mathbf{n}[e]\}$
massless	异分支+- / -+	$\{\mathbf{b}[e]\}$

同分支线含theta ordering, 称为full line; 异分支线没有theta 导数产生的shrink, 称为cross line。massive cross 仍有两个Hankel 端点态; massless cross 是单个指数乘积, 因此没有离散 n 。

3.3 Bubble 图作为输入例子

下图只是一个topology 输入例子, 不代表生成器硬编码bubble。两条内线可分别选为massive/massless full/cross; 只需改变family metadata, 统一积分Head 不变。



例如mixed bubble 可取 $Q_1 = q$ 为massive, $Q_2 = q - k$ 为massless。pure massless bubble 则把两线都设为massless。顶点符号从++ 改成+- 时, 两条线自动从full 变为cross; 不是换一套积分Head。

4 统一三槽积分表示

所有top/sub-sector 都使用三个正式槽位

$$\boxed{J[\{a_v\}, \{\text{pack}_e\}, \{n_{\text{isp}, r}\}]} \equiv J[\mathbf{aList}, \mathbf{linePacks}, \mathbf{ispList}]. \quad (4.1)$$

三个槽位分别是

1. **aList**: 当前sector 的独立时间变量幂次;

2. **linePacks**: 按固定line slot 排列的传播子/缩并线指标包;
3. **ispList**: ISP 的正式指标槽, 不是附注metadata。

实际幂次包含初始化零点:

$$A_v = a_v + a0_v, \quad B_e = b_e + b0_e, \quad B_e^S = bS_e + bS0_e. \quad (4.2)$$

质量、 ν_e 、端点、SK 符号、动量表达式、顶点能量、外不变量名称和symmetry rules 都不写入 J 。

theta 导数缩并线时, pack 变为

$$\text{pack}_e = \{bS_e\}. \quad (4.3)$$

delta 函数合并两个时间变量, 所以**aList** 只保留一个代表顶点槽; 原顶点到compact slot 的映射保存在**sectorMetadataList**。line slot 始终保留, 不能删除 bS_e 。

5 Massive 导数、EOM 与缩并

5.1 端点导数与即时EOM

massive full/cross 的两个端点态均取 $n_{e,r} = 0, 1$ 。time 导数先产生

$$\partial_{\tau_r} : \quad -J[\dots, \{b_e - 1, n_{e,r} + 1, \dots\}, \dots]. \quad (5.1)$$

若 $n_{e,r} = 1$, 二阶态必须立刻EOM。对h:

$$J[n_{e,r} = 2] = -J[n_{e,r} = 0] - (2\nu_e + 1)J[a_r - 1, b_e + 1, n_{e,r} = 1]. \quad (5.2)$$

对裸H:

$$J_H[n_{e,r} = 2] = -J_H[n_{e,r} = 0] - J_H[a_r - 1, b_e + 1, n_{e,r} = 1] + \nu_e^2 J_H[a_r - 2, b_e + 2, n_{e,r} = 0]. \quad (5.3)$$

另一端点指标不变。最后一项就是 ν_e^2/x^2 二次pole; loop-momentum 导数若产生 $n = 2$, 同样在seed 层立即使用对应EOM。当前014 由统一函数系统编译这些递推; 旧版内置公式只作为历史基线。

5.2 通用 2×2 函数系统输入

当前016 对每条massive line 输入

$$f'' + P(x)f' + Q(x)f = 0, \quad \mathbf{F} = T(x)(f, f')^T, \quad x = -\xi\tau, \quad (5.4)$$

其中 P 是一阶导系数、 Q 是零阶系数, T 缺省为单位矩阵; 还必须给原始导数基底的完整Wronskian W 。初始化层统一生成

$$A_0 = \begin{pmatrix} 0 & 1 \\ -Q & -P \end{pmatrix}, \quad A_T = T'T^{-1} + TA_0T^{-1}, \quad W_T = \det(T)W. \quad (5.5)$$

IBP 的time/radial 导数只调用由 A_T 编译的移位数据，theta shrink 只调用由 W_T 编译的数据；不在IBP 层重算 P, Q, T, W 。缺省massive 模式是h，直接取 $P_h = (2\nu + 1)/x$ 、 $Q_h = 1$ 、 $T = I_2$ 与完整 W_h ，所以 $W_T = W_h$ 。裸H 是另一个 $T = I_2$ preset。若从H 输入经一般基变换构造h，则

$$\mathbf{h} = x^{-\nu} \begin{pmatrix} 1 & 0 \\ -\nu/x & 1 \end{pmatrix} \mathbf{H}, \quad W_h = x^{-2\nu} W_H. \quad (5.6)$$

该编译接口已在当前014 实现；显式 W_T 只作 $W_T = \det(T)W$ 的一致性校验。缺省h、裸H、一般 T 和非法输入由函数系统专项检查覆盖，h/H 另有独立物理benchmark。Wronskian 的标准推导见技术笔记专门subsection。

用户输入一般函数系统时，在对应massive line 中加入

```
"functionSystem" -> <|
  "variable" -> x,
  "P" -> Pexpr,
  "Q" -> Qexpr,
  "T" -> {{t11,t12},{t21,t22}},
  "W" -> Wexpr,
  "WT" -> Automatic,
  "shrinkBShift" -> 1,
  "shrinkZeroPointShift" -> zeroShift
|>
```

也可在放入line 前调用

```
compiled = compileFunctionSystem[functionSystem];
compiled["status"]
compiled["AT"]
compiled["WT"]
```

检查编译结果。T 省略时为单位矩阵；输入必须通过可逆性、Abel/Wronskian 一致性和有限Laurent 编译门禁。

5.3 Massive theta shrink

只有massiveFull 且 $n_{e,1} + n_{e,2} = 1$ 时有Wronskian shrink。端点 r 的系数为

$$\mathcal{C}_e(-1)^{n_{e,r}+V_{\pm}}, \quad V_{\pm} = \begin{cases} 1, & ++, \\ 0, & --, \end{cases} \quad \mathcal{C}_e = \frac{4i}{\pi} e^{\pi \text{Im}\nu_e}. \quad (5.7)$$

当前实现允许line/case 显式覆盖sign offset，但默认值遵循式 (5.7)。

对h 模式：

$$a_{\text{merged}} = a_u + a_v - 1, \quad a0_{\text{merged}} = a0_u + a0_v - 2\nu_e, \quad (5.8)$$

$$bS_e = b_e + 1, \quad bS0_e = b0_e + 2\nu_e. \quad (5.9)$$

对H 模式整数移位相同，但两个 $2\nu_e$ zero-point shift 都为零。massiveCross 没有theta shrink。

6 Massless 有向单 n 与time-IBP

6.1 有序端点定义

对有序端点 $\{u, v\}$, 令 $\Delta = \tau_u - \tau_v$ 。masslessFull 的同分支符号记为 $\sigma = +1$ 对 $++$ 、 $\sigma = -1$ 对 $--$:

$$M_0 = \theta(\Delta)e^{-i\sigma q\Delta} + \theta(-\Delta)e^{+i\sigma q\Delta}, \quad (6.1)$$

$$M_1 = -\theta(\Delta)e^{-i\sigma q\Delta} + \theta(-\Delta)e^{+i\sigma q\Delta}. \quad (6.2)$$

第一端点定义 M_1 的方向。交换端点时 M_0 不变、 M_1 变号。若暂用旧双端点标签做中间推导,

$$\{10\} = -\{01\}, \quad \{20\} = \{02\} = -q^2\{00\}, \quad \{11\} = +q^2\{00\}. \quad (6.3)$$

正式 J 从不保存这些双端点标签, 只使用 $\{b_e, n_e\}$, 其中 $n_e = 0, 1$ 。

每条massless line 仍保留自己的 $\{b_e, n_e\}$ pack; 只有共享同一当前代表顶点对的time boundary 按共同theta 组合。

6.2 两个端点的regular 与delta 规则

由式 (6.2),

$$\partial_{\tau_u} M_n = +i\sigma q M_{1-n} - 2n \delta(\tau_u - \tau_v), \quad (6.4)$$

$$\partial_{\tau_v} M_n = -i\sigma q M_{1-n} + 2n \delta(\tau_u - \tau_v). \quad (6.5)$$

所以四个基本端点检查为

n	端点	regular 指标变化与系数
0	第一端点	$+i\sigma \{b-1, 1\}$
0	第二端点	$-i\sigma \{b-1, 1\}$
1	第一端点	$+i\sigma \{b-1, 0\}$
1	第二端点	$-i\sigma \{b-1, 0\}$

同一端点连续作用两次, regular 部分回到原 n :

$$\partial_{\tau_u}^2 : -J[\{b-2, n\}], \quad \partial_{\tau_v}^2 : -J[\{b-2, n\}]. \quad (6.6)$$

6.3 Massless shrink 与抵消

只有 $n = 1$ 产生theta-delta:

$$\partial_{\tau_u} : -2\delta(\tau_u - \tau_v), \quad \partial_{\tau_v} : +2\delta(\tau_u - \tau_v). \quad (6.7)$$

massless shrink 没有Hankel Wronskian prefactor, 也没有 2ν shift:

$$a_{\text{merged}} = a_u + a_v, \quad a0_{\text{merged}} = a0_u + a0_v, \quad bS_e = b_e, \quad bS0_e = b0_e. \quad (6.8)$$

缩并后, 剩余传播子的两个原端点可能映到同一个active vertex。此时:

1. time 导数必须同时作用于两个端点，不能只取第一个匹配；
2. 两个regular 端点贡献按式 (6.5) 抵消；
3. 两个theta-delta 贡献的 $-2/ + 2$ 系数相消，不再产生shrink 项；
4. 反对称态 $n = 1$ 在coincident endpoint 恒为零。

这个零关系也必须作用于从top 方程产生的目标sub-sector 项。当前实现根据每个 J 中已经变成单元素pack 的shrunk lines，重建目标sector 的顶点代表映射，再判定coincident $n = 1$ ；不能只读取source top 的endpoints。

对 $+-$ 或 $-+$ masslessCross，没有 n 、theta 或shrink。端点regular 系数由该端点分支决定：

$$+\text{端点} : +i \{b - 1\}, \quad -\text{端点} : -i \{b - 1\}. \quad (6.9)$$

6.4 同一顶点对多条full lines

同一当前代表顶点对的massive/massless full lines 共享时间差 Δ 。写

$$G_e = \theta(\Delta)A_e + \theta(-\Delta)B_e, \quad (6.10)$$

则整个bundle 的boundary 只有

$$\delta(\Delta) \left(\prod_e A_e - \prod_e B_e \right). \quad (6.11)$$

定义 $J_e = (A_e + B_e)/2$ 、 $D_e = A_e - B_e$ ，可保持现有逐线pack：

$$\prod_e A_e - \prod_e B_e = \sum_{\substack{\emptyset \neq S \subseteq B \\ |S| \text{ odd}}} 2^{1-|S|} \prod_{e \in S} D_e \prod_{e \notin S} J_e. \quad (6.12)$$

一次事件可同时shrink 任意非空奇数条bundle 线，系数为 $2^{1-|S|}$ ，但只合并一次顶点且只含一个delta。两线只有single contacts；三线另有系数 $1/4$ 的triple contact。未选择的coincident full lines 立即应用massless $n = 1$ 为零和massive endpoint-swap canonical，不会再次产生theta boundary。

sector 只允许连接两个不同当前代表类的contact 事件，按BFS 枚举；事件之间形成forest，但单个事件可选择多条平行线。单传播子的对称equal-time 值取 $\theta(0) = 1/2$ ，但不把它作为 $\delta\theta^m$ 的点值乘法规则。保留逐线theta 时，统一Gaussian mollifier 给出的线别积分在求和后严格等价于式 (6.12)。

例如三条同向masslessFull 线均取 $n_1 = n_2 = n_3 = 1$ ，并从第一有序端点求导。此时每条线的对称等时因子为零、contact difference 为 -2 ，所以三个single-contact 项都因未选中线的等时因子为零而消失，只剩

$$\frac{1}{4}(-2)^3 = -2. \quad (6.13)$$

指标上即

$$J[\{a_u, a_v\}, \{\{b_1, 1\}, \{b_2, 1\}, \{b_3, 1\}\}, \{\cdots\}] \longrightarrow -2J[\{a_u + a_v\}, \{\{b_1\}, \{b_2\}, \{b_3\}\}, \{\cdots\}], \quad (6.14)$$

三个pack 同时shrink，但只合并一次顶点且没有额外massless 幂次shift。共同-theta与Gaussian 逐线方案的完整推导、massive W_T 例子和等价条件见技术笔记附录。

7 Topology、标量积、ISP 与外腿能量

7.1 输入骨架

```
case = <|
  "name" -> "mixedBubble",
  "vertexData" -> {{v1,"+"},{v2,"+"}},
  "lineData" -> {
    <|"id"->1,"endpoints"->{v1,v2},"momentum"->q,
      "nu"->nuM,"bbType"->"h","massType"->"massive"|>,
    <|"id"->2,"endpoints"->{v1,v2},"momentum"->q-k,
      "nu"->0,"bbType"->"exp","massType"->"massless"|>
  },
  "loopMomenta" -> {q},
  "loopExternalMomenta" -> {k},
  "independentExternalMomenta" -> {kE},
  "ibpMode" -> "full",
  "vertexEnergies" -> <|v1->Sqrt[sp[kE,kE]],v2->E2|>,
  "ispData" -> {},
  "zeroPointRules" -> {
    a0[v1]->alpha1,a0[v2]->alpha2,
    b0[1]->beta1,b0[2]->beta2
  },
  "numericRules" -> {dim->3,ss11->5,sE1->7,nuM->2},
  "symmetryRules" -> {}
|>;
```

`endpoints` 对 `masslessFull` 是有序输入；交换顺序会改变 $n = 1$ 表示的符号。

输入后推荐立即通过公开初始化入口运行

```
context = DSInit[case, GenerateDerivativeMetadata -> True];
topo = context["topology"];
validation = context["validation"];
derivatives = context["derivatives"];
```

`validation["issues"]` 给出 `topology/ISP/函数系统` 问题；初始化 `metadata` 同时保存 `sector`、`convention` 和可选微分算符信息。

7.2 sp 与外不变量

用户端所有圈动量相关标量积统一写 `sp[p,r]`。代码直接使用

```
SetAttributes[sp, Orderless];
```

实现 $sp(p, r) = sp(r, p)$ 。这只是标量积交换性，不是图或积分族对称性。

用户可给圈动量和外动量任意符号名，例如 `sah, bob, alice`。016 不从符号字符串猜测角色，也不再从一个统一列表自动分类；输入必须显式声明

loopMomenta
loopExternalMomenta
independentExternalMomenta

loopExternalMomenta 是圈积分外动量基，决定完整Gram 坐标、 dqk 生成元和ISP 闭合维数；independentExternalMomenta 是topology 中实际无圈line/phase 的独立模长列表，只定义 $|P_e|$ ，不定义彼此点积。旧字段externalMomenta/externalLegMomenta 仅作为分别对应这两个字段的兼容别名；同时给新旧字段时以新字段为准。程序会从line routing 独立计算实际需要的空间并与用户声明比较，而不会替用户选择基。

本手册以后把loopExternalMomenta 的第 i 项简称为 kLi ，把independentExternalMomenta 的第 e 项简称为 kEe 。这是按两个输入列表的位置定义的角色名，不要求用户真的把动量符号写成 $kL1/kE1$ ；alice、bob 等任意符号同样合法。两套编号各自从1 开始且互不共享： kL 第 i, j 项在内部形成 $qk[*, i]$ 、 $kk[i, j]$ ，公开为 $ssij$ ； kE 第 e 项的模长平方在坐标/Jacobian 层使用externalLegSquaredCoordinate[e]，公开为 sEe ，不进入loop-IBP/ISP 的 K 。

结构圈数由内部无向多重图的第一Betti 数

$$L = E - V + C \quad (7.1)$$

给出；平行边逐条计数，自环各贡献一圈，extLegs 不计入 E 。逐边删除后连通分量增加者为bridge，其余为cycle line。ibpMode->"full" 还要求loopMomenta 数量等于 L 、routing 满秩并位于incidence cycle space；ibpMode->"timeOnly" 不生成momentum IBP，树图缺省即为该模式。

为消除任意圈动量平移，程序把line routing 写成 $Q = Aq + r$ ，选满秩参考行 A_R 后使用

$$q' = A_R q + r_R, \quad r' = r - A A_R^{-1} r_R. \quad (7.2)$$

因此 $\{q+k1, q+k1+k2\}$ 实际只要求方向 $k2$ ；加减号始终保留精确系数。这个计算只用于审计用户列表是否完备，不用于自动改写用户选定的动量基。

016 内部保留用户给定loop-external 基的平方Gram 原子，公开缺省为根号坐标：

```
kk[i,j] = sp[ki,kj]
ssij = Sqrt[sp[ki,kj]]
```

用户输出不保留内部 $kk[i, j]$ 。

7.3 ISP 完备性

L 圈、 K 个独立外动量共有

$$N_{sp} = \frac{L(L+1)}{2} + LK \quad (7.3)$$

个圈相关独立标量积。用户定义的propagator 模长平方和ISP 必须构成可反解坐标。ISP 可以是一般标量积，例如

```
sp[l3,k321+l3]
```

而不只限于 $q_i \cdot q_j$ 。package 验证闭合性，但不自动删线或替用户重选propagator basis。

每个ISP 使用name/expr/range Association，或等价三元组{name,expr,range}：

```
"ispData" -> {
  <|"name"->rho1,"expr"->sp[q1,k],"range"->{0,1}|>,
  <|"name"->rho2,"expr"->sp[q2,k],"range"->{0,1}|>
}
```

name 必须唯一，**expr** 使用已声明的动量基，**range** 给出numerator 指标seed 值域。独立benchmark 与package 使用同一schema。

7.4 实际无圈模长与fixed line

independentExternalMomenta 中每个表达式定义一个实际无圈模长。程序要求它们覆盖所有active 无圈line/被审计phase 中不能由完整loop Gram 表示的模长；不会从向量原子表自动拆分或补齐。

程序把实际无圈模长平方在完整loop Gram 与此前已选无圈平方坐标上按首次出现顺序做秩筛选，只为独立项建立模长变量：

```
sE1 = Sqrt[sp[PE1,PE1]], sE2 = Sqrt[sp[PE2,PE2]], ...
```

未入基的实际出现项保存为当前平方坐标的dependent binding；这些模长不进入loop IBP generator 或ISP closure。与两组动量都无关、只进入顶点相位的能量仍使用独立标量 $ke[i]$ 。

例如

```
"loopExternalMomenta" -> {k1,k2}
"independentExternalMomenta" -> {kE1,kE2,kE1+kE2}
"lineData" -> {...,"momentum"->kE1,...}
"vertexEnergies" -> <|v1->Sqrt[sp[kE1+kE2,kE1+kE2]],...|>
(* loop Gram: {ss11,ss12,ss22}; no-loop magnitudes: {sE1,sE2,sE3} *)
```

无圈模长只把整体反号 $P \leftrightarrow -P$ 视为同一对象； $|alice+bob|$ 与 $|alice-bob|$ 必须分别声明，不能合并。

坐标名只由两个用户列表各自的稳定顺序决定，与符号名无关。每套列表的总数为1-9 时使用ss11/sE1；10-99 时使用两位补零，例如ss0101,ss0110,sE01；100 起按该列表总数位数自然扩展。若显式给出旧平方坐标规则，仍以单位Jacobian 工作，但它不再是016 缺省：

```
"externalInvariantRules" ->{sp[k1,k1]->s11,...}
```

特别地，若topology 实际分别出现 $|k_{E1}|$ 、 $|k_{E1}+k_{E2}|$ 、 $|k_{E2}|$ ，用户就应在**independentExternalMomenta** 中给出这三个表达式，缺省得到sE1,sE2,sE3；程序既不输出sp[kE1,kE2]，也不用余弦定理消去中间变量。若再声明 $2k_{E1}$ ，它是过完备项并触发warning。

在loop 表示中，只有cycle line 的pack 含b/bS；bridge/non-cycle line pack 只含endpoint n。fixed line 的零点幂和导数产生的幂变化全部进入显式模长系数。**timeOnly** 下所有active lines 都使用fixed-coefficient schema；即使输入图含结构圈，也不为line 伪造b/bS。但**timeOnly** 不等于一律使用单槽tree 表示：程序根据离散态能否无损放入vertex pack 自动选择两条公开路线，详见第 9.1 节。

初始化采用不阻塞脚本的两阶段接口：

```

proposal = DSKinematics[case];
(* 查看 proposal["defaultRules"] 与 proposal["selectionTemplate"] *)
custom = {sp[k1,k1]->u^2, sp[k1,k2]->u v,
          sp[k2,k2]->v^2+w^2, ...};
audit = DSKinematics[case, custom];
context = DSInit[case, KinematicRules->custom];

```

DSKinematics 的返回值分组列出：结构图/routing 审计、两个用户列表的exact/over/under 状态、基础坐标顺序、动力学规则矩阵与参数Jacobian、零空间和capability。主要Association key 包括

```

momentumDeclarationStatus, kinematicCoordinateStatus,
dependentMagnitudeBindings, missingDirections, missingMagnitudeSquares,
capabilities.

```

欠完备动量列表或动力学规则会以红色error 显示缺失方向/零空间并令DSInit 返回失败；所有后续模块读取同一capability，不能继续生成seed。过完备以红色warning 提醒，允许初始化和symbolic IBP seed，但关闭ds/DSDE 与唯一rep2innerform。对于过完备loop 声明，用户原列表保留作审计，实际 n_K 、Gram、dqk 与ISP 闭合只使用affine quotient 必要基effectiveLoopExternalMomenta；共同loop shift 不会虚增标量积空间。只有exact 状态同时开放初始化、time/momentum IBP、求导和反变换。

成功初始化后可直接查看或重定义notation:

```

notation = DSParameterNotation[context];
newContext = DSRedefineParameters[context, {
  sp[k,k] -> loopScale^2,
  sp[p1,p1] -> leg1^2,
  sp[p2,p2] -> leg2^2,
  sp[p1+p2,p1+p2] -> leg12^2
}];

```

新规则必须完备；返回的新context 有新的审计和inputHash，后续seed、ds 与DSDE 都读取它。规则右端可以是一般混合参数化，例如 $(u+v)^2$ ；程序先展开每条右端并提取原子参数 u, v ，再建立完整Jacobian。参数数多于基础坐标数时属于过完备并关闭求导。正式函数名是DSRedefineParameters。

7.5 独立变量与微分方程求导

当前016 提供seed 层和表达式层的独立变量求导接口。变量分三类：

- 圈线相关外动量模长，缺省为ssij，通过外动量矢量导数组合实现。
- 实际无圈动量模长独立基，缺省为sE1,sE2,...；从属模长先写成这些变量与ssij 的binding。若它对应一条传播子，导数按binding Jacobian 同时作用于该线的分母和massive/massless building block。
- 独立顶点能量参数，例如ke[i]；它不是动量向量，不定义点积。

对顶点能量 y ,

$$\frac{\partial}{\partial y} J = \sum_v \left[-\eta_v \frac{\partial E_v}{\partial y} \right] J[a_v \rightarrow a_v + 1], \quad \eta_v = \begin{cases} -i, & v = +, \\ +i, & v = -, \end{cases} \quad (7.4)$$

其中 $E_v = \text{vertexExternalEnergy}[\text{topo}, v]$ 。额外负号来自本package的时间幂convention $(-\tau_v)^{a_v+a_0v}$: 相对 E_v 求导产生 $\eta_v \tau_v = -\eta_v(-\tau_v)$ 。

对外不变量 x_a , 先引入外动量矢量导数

$$D_{ij} = k_i \cdot \frac{\partial}{\partial k_j}, \quad D_{ij}(k_m \cdot k_n) = \delta_{jm}(k_i \cdot k_n) + \delta_{jn}(k_i \cdot k_m). \quad (7.5)$$

然后在选定外不变量坐标 $\{x_b\}$ 上解约束方程

$$\frac{\partial}{\partial x_a} = \sum_{ij} c_{ij}^{(a)} D_{ij}, \quad \sum_{ij} c_{ij}^{(a)} D_{ij} x_b = \delta_{ab}. \quad (7.6)$$

完整 K^2 个 D_{ij} 通常含零空间, 解不唯一; 当前实现默认使用上三角external-vector basis 取一组canonical 解, 并在decomposition 数据中显式返回矩阵、系数、残差、nullity 与nonUniqueQ。

相关接口为

```
makeExternalInvariantDerivativeDecomposition[topo, var]
applyExternalVectorDerivativeSeed[topo, int, gen]
applyExternalInvariantVariableDerivativeSeed[topo, int, var]
applyIndependentVariableDerivativeSeed[topo, int, var]
ds[expr, ssij, context] (* context["topology"] 也可 *)
ds[expr, ssij]
```

applyIndependentVariableDerivativeSeed 会自动分派: loop Gram 变量先做上述decomposition, 再把每个 D_{ij} 作用到传播子、building block、ISP 和相位; 实际无圈模长及其从属binding 使用line-local 径向导数并微分相位; 其它变量按独立顶点能量标量处理。一般用户坐标 u 直接使用 $\sum_a (\partial x_a / \partial u) \partial_{x_a}$, 不会因 u 同时进入多个Gram 原子或binding 而提前返回。

ds 是表达式级总导数入口。变量必须使用函数族初始化后的外部名字; 内部kk[i,j] 不接受。若

$$\mathcal{I}(s) = \sum_r c_r(s) J_r(s) + c_0(s), \quad (7.7)$$

则

$$\partial_s \mathcal{I} = \sum_r c'_r(s) J_r(s) + \sum_r c_r(s) \partial_s J_r + c'_0(s). \quad (7.8)$$

程序先将 J_r 替换为惰性token 求显式系数导数, 再补上每个积分的指标导数, 合并后统一执行EOM、symmetry 与canonical。它接受单个 J 和任意线性组合, 拒绝 $J_i J_j$ 等非线性输入。

令 $x_{ij} = \text{sp}(k_i, k_j) = \text{ssij}^2$ 。016 不重写平方Gram 导数, 而是使用 $\partial_{\text{ssij}} = 2 \text{ssij} \partial_{x_{ij}}$ 调用原子层。显式系数仍执行完整乘积与链式法则; 程序不使用PowerExpand。

对无圈massive 线 e , 记 $B_e = b_e + b_{0e}$ 。径向导数包含 $-B_e J[b_e \rightarrow b_e + 1]$; 若 A_T 的某个Laurent 项为 $c x^p$ 并把端点态 α 送到 β , 则另有

$$c J[a_v \rightarrow a_v + p + 1, b_e \rightarrow b_e - p, n_{e,v} : \alpha \rightarrow \beta]. \quad (7.9)$$

这直接读取与IBP 相同的`derivativeTerms`, 不读取只属于time-theta shrink 的`WT/shrinkTerms`。

当前验证还包括两个表达式级benchmark: 十个物理函数族在全部sign/mode、可达sector 和独立变量上的general-index 总导数为468/468; 旧reference bubble 在对齐 $G/R1/R2$ 、 $V_{pm} = 0$ 、zero-point、symmetry、parity 和 $s_{11} = k_s^2$ 后为80/80。两者都使用带动力学量系数的两个积分及纯系数项, 而不是只检查单积分导数。

对massive building block, external-vector/外不变量导数与qIBP、tIBP 自动使用同一份line-local `compiledFunctionSystem`。普通导数只读取最终 A_T 编译的`derivativeTerms`, 因此用户选择h、裸H 或一般 P, Q, T, W 后不需要为微分方程另设convention。 $W_T = \det(T)W$ 及其`shrinkTerms` 只用于time-IBP 的theta coincidence/Wronskian shrink; 普通动力学量导数不产生theta shrink, 因而不调用 W_T 。

7.6 用户对称性规则

一般积分族对称性依赖质量和外参条件, 当前实现不自动猜测一般图automorphism。程序会自动加入受门禁保护的原始self-loop tadpole 规则, 并与用户`symmetryRules` 取去重并集:

```
repSymmetry0[topo_]
effectiveSymmetryRules0[topo_]
symmetry[expr_, topo_]
```

`repSymmetry0` 只返回用户原始规则; `symmetry` 对并集只做一次替换。自动规则只识别原始端点相同的self-loop, 不作用于cross propagator 或shrink 后才coincident 的普通线; odd ISP 归零还要求对应loop momentum 不被其它传播子共享。

8 IBP 生成元与公开接口状态

8.1 完整生成元

每个active vertex 有一个time generator。对 L 个圈动量和 K 个`loopExternalMomenta`, loop-momentum generators 为

$$\mathcal{O}_{\ell,v} = \frac{\partial}{\partial q_\ell^\mu} v^\mu, \quad v \in \{q_1, \dots, q_L, k_1, \dots, k_K\}, \quad (8.1)$$

总数 $L(L + K)$ 。不能漏掉 $d/dq_\ell \cdot q_m$ 的交叉项或 $d/dq_\ell \cdot k_j$ 。

当前核心使用`applyTimeGeneratorSeed` 与`applyMomentumGeneratorSeed`。微分方程方向的外部变量求导使用上一节列出的`applyIndependentVariableDerivativeSeed` 等接口。自动batch 必须在给定离散 n 后调用, 再立即EOM/canonical。

8.2 公开原子API

016 保留以下原子接口。推荐显式传入DSInit context 或parsed topology; 若先调用setIBPTopologyContext 也可使用短签名:

```
dtau[i,expr,context]      or dtau[i,expr]
dqq[i,j,expr,context]    or dqq[i,j,expr]
dqq[i,j,expr,context]    or dqq[i,j,expr]
ds[expr,var,context]      or ds[expr,var]
rep2innerform[expr,context]
rep2outform[expr,context]
rep2Integrand[expr,context]
```

上式第三参数也可写成context["topology"]。resolver 会先识别完整context 并解包, 不会把它误作原始topology case 再次解析。前三个复合算子分别表示 $d/d\tau_i$ 、 $d/dq_i \cdot q_j$ 、 $d/dq_i \cdot k_j$, 对 J 的线性组合保持线性, 并复用现有EOM、theta boundary 与canonical。

例如先把离散态替换为字面0/1, 再调用

```
int = makeBaseIntegral[topo] /. seedRules;
timeRelation = dtau[v1,int,topo];
qqRelation = dqq[1,1,int,topo];
qkRelation = dqq[1,1,int,topo];
```

```
setIBPTopologyContext[topo];
sameTimeRelation = dtau[v1,int];
clearIBPTopologyContext[];
```

短签名依赖唯一的已注册topology; 同时处理多个topology 时应始终使用显式参数。

rep2innerform/rep2outform 转换用户动量与内部编号、sp 与qq/qk/kk、外不变量名和ISP 名。实现会先保护所有 J , 因此不改变三个指标槽、line pack 次序或任何 $a/b/n/ispN$ 值。

rep2Integrand 把 J 展开为被积函数。普通时间幂、顶点相位、分母幂和ISP 直接相乘; 每条未缩并传播子的非平凡Hankel/指数/theta block 用惰性Hh[...] 包装。它不积分、不生成IBP、不应用EOM。

9 Seed、linearData 与serializer

canonical seed 必须满足:

- 每个实际sector 都有全部active time 和 $L(L + K)$ momentum generators;
- massive 无 $n \geq 2$;
- masslessFull 只有 $n = 0, 1$;
- 共同-theta odd-subset contact、contact 可达sector 与coincident canonical 已处理;
- 解析seed 保留非零 $a0/b0/bS0$ 。

9.1 timeOnly 的两种公开表示

DSSeeds 对timeOnly family 自动选择表示, 不要求也不接受用户传入私有的state selector:

- 若不存在未缩并的masslessFull line, binary massive 状态可以按顶点打包, 返回representation $\rightarrow$ "J[vertexPacks]"; 这是DSTreeSeeds、repIterative 与两条tree DE 路线使用的direct pure-time 表示。
- 若存在未缩并的masslessFull line, 单个有向状态 n_e 属于整条line, 而不是某一个顶点。此时返回representation $\rightarrow$ "J[timePowers,linePacks,isp]", 并在fixed line pack 中保留{n[e]}; 缩并后该pack 为{}。这种表示仍没有b/bS、ISP 或momentum generator。

例如atomic massless 同分支family 的公开流程为

```
context = DSInit[case, RegisterAsCurrent->False];
seeds = DSSeeds[context, DiscreteMode->"all"];
linear = DSLinear[seeds, context];
```

其中top sector 的状态集合为 $n_1 \in \{0,1\}$, 同分支可达sector 为{"top","e1"}, 异分支只有{"top"} 且line pack 为{}。两个顶点各自的time generator 作用于top 的两个状态; 同分支shrink sector 另有合并顶点的time seed。成功时DSSeeds 与DSLinear 都返回status $\rightarrow$ "generated"。DSLinear 会按representation 自动消费direct vertex-pack 或canonical line-pack batch; 调用者不得把单槽tree 规则强套到massless line-pack batch。

linearData 是后端中立的Mathematica Association, 不是reduction 结果。核心字段为:

```
linearEquations, integralList, integralRules
sectorMetadataList, readiness reports
```

全sector 主积分候选必须统一排序, 不能按sector 依次追加。

serializer 只转换linearData 的编号和语法。Kira serializer 默认只写基础输入、映射和metadata, 包括ibp.kira、list 与jobs.yaml。它不保存本机Kira/Fermat 路径, 也不运行Kira。

统一seedRanges 给出family 的缺省连续范围。需要复现非矩形reference seed 时, 可在输入中增加generatorSeedRanges, 按sectorKey 与generator label 只覆盖指定变量; 未列出的变量继续使用统一范围。每个generator 的最终变量顺序、value lists、配置范围、规则数、方程数与range source 都保存在batch metadata 中。

完整应用方式为

```
batch = makeCanonicalSeedBatch[
  topo,
  DiscreteMode -> "all",
  GenerateShrinkSectors -> True
];
```

```

seedSave = writeSeedBatchMMA[
  batch,
  OutputDirectory -> "output/seeds"
];

linearData = makeLinearSystemData[batch,topo];
sampled = makeSampledLinearSystemData[
  batch,topo,
  CoefficientRules -> {dim->3,ss11->5,nuM->2}
];

kiraPreview = makeKiraExportData[sampled];
kiraFiles = makeKiraExportData[
  sampled,
  OutputDirectory -> "output/kira"
];

```

`makeKiraExportData` 只接受由完整canonical batch 得到的linear-system 数据; seed batch 不能直接导出Kira。

也可用一站式接口串联门禁:

```

workflow = makeIBPWorkflowData[
  case,
  DiscreteMode -> "all",
  LinearSystemMode -> "symbolic",
  ExportKira -> False
];
ready = makeIBPReadinessReport[
  case,
  DiscreteMode -> "all",
  LinearSystemMode -> "symbolic"
];

```

`LinearSystemMode` 可取"symbolic"、"sampled" 或"numeric"; numeric 模式要求`numericRules` 覆盖报告中列出的外不变量、顶点能量和线参数。

10 016 标准package 与初始化

一个016 example 的开头分为四个独立单元: package 加载、详细输入、缺省option、初始化。所有路径相对于example 的main.wl:

```

exampleDir = DirectoryName[$InputFileName];
packageDir = FileNameJoin[{
  exampleDir,"..","..","..","000_code","016_dSIBP"
}];

```

```

If[!MemberQ[$Path,packageDir],AppendTo[$Path,packageDir]];
Needs["dSIBP"];

(* 详细输入: vertexData, lineData, momenta, ISP, zero point,
   symmetry, parity 和 independent variables. *)
case = <| ... |>;

(* 缺省选项: WriteInitializationFiles->False;
   GenerateDerivativeMetadata->False; OverwriteInitialization->False. *)

init = DSInit[case,
  InitializationDirectory->FileNameJoin[{exampleDir,"init"}],
  WriteInitializationFiles->True,
  GenerateDerivativeMetadata->True
];

```

init/ 中的manifest.wl、topology.wl、sectors.wl 和conventions.wl 是可重新Get 的Association。derivatives.wl 缺省不生成；只有显式打开GenerateDerivativeMetadata 时才写出。已有目录若对应不同输入哈希，缺省拒绝覆盖。

DSSeeds 的公开缺省为DiscreteMode->Automatic。它读取初始化topology 的seed preset: 缺省quickCheck 解析为"sample", fullDiscrete 与bounded 解析为"all"。正式全离散验证应显式传"all", 避免调用者把快速检查误解为完整覆盖。timeOnly 的表示由topology line state 自动选择；不存在需要用户填写的TreeState 或其它私有分派参数。

11 016 运行消息与进度

016 使用一个集中消息层，所有耗时模块只上报结构化事件，不在内部循环散落Print:

```

DSMessagesOn[]
DSMessagesOff[]
DSMessagesQ[]

```

缺省开启info、progress 和非致命warning。关闭后隐藏这三类提醒，但fatal error 永远不能静默；函数返回值中的status/issues 仍是权威状态。warning 在notebook 中使用橙色，error 使用红色粗体，同时都发出标准Mathematica Message，使headless 日志也能捕获。

sector 初始化、偏导算符生成、seed、linearization、Kira 导出/导入验证、逐master/变量构造DE 和scaling check 都应给出阶段开始/结束信息。

Notebook 前端使用Monitor 配合ProgressIndicator[n,{0,m}], 或用单个PrintTemporary 配合Dynamic，在原位显示n/m。没有FrontEnd 时只打印开始、结束及大约每10% 的稀疏里程碑，禁止每处理一项增加一行。总数不能预先可靠获得的阶段只报告阶段状态，不伪造n/m。

12 016 Kira 结果取回到微分方程

package 只生成Kira 输入，不运行reduction。完整闭环为：

```
seeds = DSSeeds[init];
linear = DSLinear[seeds,init];
kiraExport = DSKiraExport[linear,
  OutputDirectory->FileNameJoin[{exampleDir,"kira"}]
];

(* 用户在 package 外运行 Kira。 *)
kiraResult = DSKiraImport[
  FileNameJoin[{exampleDir,"kira"}],init
];
de = DSDE[kiraResult,{ss11,P0},
  OutputDirectory->FileNameJoin[{exampleDir,"results","dlogDE"}]
];
scale = DSScaleCheck[de,<|
  "relation"->"PureMassiveBubble",
  "variables"->{ss11,P0},"weights"->{1,1}
|>];
```

DSKiraImport 检查export manifest、双向积分映射、master list 和完整reduction rules。任何缺失或convention/parity 不一致都会阻止DE。根号代数系数可在Kira 侧映射为小写原子，manifest 保存可逆映射；导入后恢复原表达式。DSDE 对同序master list 调用ds，因此动力学量系数的导数自动包含在内；约化后的内部kk/ISP 系数还会统一转回初始化声明的根号坐标。未约回master 的对象必须在结果中显式列出。

首个闭环example 固定使用pure massive bubble reference 的— branch 和even-parity closed subsystem，不生成全parity 大系统。reference 的顶点交换symmetry 只在 $P_1 = P_2$ 成立。reference 的 $V_{\pm} = 0$ 与package 的— 分支使用相反的能量参数号，因此package 取 $P_0 = +ik_0$ ，reference basis 映射为 $P_1 = P_2 = -P_0 = -ik_0$ ；独立 P_1/P_2 family 不得复用该symmetry。fresh Kira 2.3 运行得到33581 条方程、6555 条独立关系和19 个active masters，1814 个目标全部约化。post_kira_check.wl 对导入、master 顺序、辅助ID、DE closure、固定参数残留和Eq. (51)/(64) 的符号Euler residual 共检查22 项，全部通过。DE matrix 与完全同序的master list 一起写入results/dlogDE/。

13 Tree vertex family

Tree 与loop 使用同一个Head，但direct massive-only tree 是单槽表示：

$$J[\{\{a_1, n_{11}, \dots, n_{1p_1}\}, \{a_2, n_{21}, \dots, n_{2p_2}\}, \dots\}]$$

第 e 个pack 的第一项是时间幂次偏移 a_e ，后续 p_e 项是该顶点massive h 外腿的 $n = 0, 1$ 状态。pack 长度必须为 $1 + p_e$ ；massless 外腿不占vertex-pack 的 n 槽。因此含未缩并masslessFull 的timeOnly family 使用第 9.1 节的三槽fixed-line 表示，而不是丢掉

其有向 n_e 。三槽J[aList,linePacks,ispList]与单槽J[vertexPacks]由shape validator分开，不能交叉调用。

Loop到tree的投影使用完整物理幂次。目标项相对参考seed的时间相位为 $(-1)^{\Delta \Sigma(a+a_0)}$ ，每条线的显式能量系数为 $k_e^{-\Delta(b+b_0)}$ ；缩并线改用 $bS + bS_0$ 。当前sector的 a_0 进入tree的 ν_0 。所以h-mode contact的 $bS = b + 1$ 、 $bS_0 = b_0 + 2\nu$ 必须共同给出 $(-k)^{-2\nu-1}$ ，不能只保留整数 k^{-1} 。

单顶点 p -fold family的 2^p 个masters按二进制顺序00...0,00...1,...,11...1保存。repIterative[expr,end]使用2401.00129 Eq. (3.37)–(3.47)–(3.50)直接生成的general-index单次规则，把每个 a_e 约化到指定终点；Automatic表示全零终点，长度不符或步数不可判定时拒绝。含contact source时，raw与sector-tagged迭代都从DSTreeSeeds的direct pure-time relation生成同一首步，因而保留显式treeEnergy；旧三槽loop投影只作交叉验证。

DSTreeDLogDE的sector对角块使用该文Eq. (3.54)–(3.55)，非对角块使用Eq. (3.66)–(3.68)。letters按顶点顺序逐块排列：每个顶点先列massive-leg energies，再按binary master order列cut letters，最后稳定去重；letterMatrices使用同一键顺序。 G^{+-}/G^{-+} 没有theta，不使用Wronskian/contact； G^{++}/G^{--} 的contact source先由已验证的loop dtau、compiled W_T 、共同-theta和coincident canonical处理，再映射到tree lower sector。

程序对每个binary master保持 n 不变，只把当前求导顶点设为 $a = 1$ ，用所得 $R^{(1)}$ 构造非对角contact selector；不会误用 $a = 0$ seed的 $R^{(0)}$ 。每个lower sector自动保存共同normalization，吸收Wronskian和zero-point产生的显式能量幂；其它入边除去normalization后必须与能量无关。以下返回字段用于审计：

sectorNormalizations, normalizationAudits, contactMaps
omegaBlocks, dlogQ

masters中每项的coefficient正是对应sector normalization；矩阵、letters与该master顺序完全一致。

016 还提供不使用上述直接dlog/递推矩阵的交叉路线：

```
ibp = DSTreeNaiveIBP[context, treeDLog["masters"]];
de = DSTreeNaiveDE[ibp, {K1,K2,k12}];
```

DSTreeNaiveIBP对每个sector的binary master和活动顶点生成 $a_v = 1$ 的loop代表元，只调用dtau与既有contact canonical，再把投影方程作为有限线性系统直接求解；它不调用repIterative或 $A_{\pm}$ 。DSTreeNaiveDE对顶点相位项复用loop投影，对treeEnergy则直接使用h的动量导数关系，因为loop适配器中的内部动量是积分变量，不能冒充tree的外部能量。对于endpoint state n ，raw tree导数为

$$\partial_k J[a, n = 0] = -J[a + 1, n = 1], \quad (13.1)$$

$$\partial_k J[a, n = 1] = J[a + 1, n = 0] - \frac{2\nu + 1}{k} J[a, n = 1]. \quad (13.2)$$

同一传播子的两个endpoint分别贡献，随后统一用naive time-IBP约到指定masters。

若master定义为 $N_s J_s$ ，乘积法则中的 $(\partial N_s) J_s$ 会单独加入；输出的matrices/sources/residualObjects与输入master顺序一致。缺省masters来自DSTreeDLogDE；做独立检验时应显式传入列表，确保两路线只共享basis和normalization，而不共享reduction rules。

若不同lower sectors 具有相同pack shape, 裸单槽J 无法单独携带sector 身份。014 的内部tree linear data 和约化token 保存sector key; 公开per-sector 积分仍使用同一Head J, 不另建并行表示。

14 常见错误与当前限制

- 把sp 的Orderless 当作图对称性: 错误。
- massless line 交换endpoints 却不翻转 $n = 1$ 表示: 错误。
- massless $n = 1$ theta 导数漏掉 $-2/ + 2$ shrink: 错误。
- 对 $\delta\theta^m$ 直接代入 $\theta(0)^m = 2^{-m}$: 错误; 必须先合并共同theta 或统一正则化整个乘积。
- massive/massless shrink 都令merged a 减1: 错误; 只有massive Wronskian 有该整数移位。
- 把所有full lines 的幂集当作sector: 错误; 只枚举contact 可达状态。
- top 方程的sub-sector 项仍用source endpoints canonical: 错误, 当前实现已按目标sector 修复。
- massive ++ 与-- 使用同一Wronskian offset: 错误; 当前实现使用各自正确的offset。
- 在EOM 前保留符号 n 或把 $n = 2$ 留给后端: 错误。
- shrink 后仍在 J 保留两个独立 a : 错误。
- 把ke[i] 放入momentum generators: 错误。
- seed 直接导出Kira: 错误; 先构造linearData。

当前未实现: 一般图automorphism/参数对称性的自动发现、scaleless/parity 全自动前端、自动选择propagator basis、streaming/chunked 高圈seed, 以及实际运行reduction。016 已实现标准context、动力学变量审计、交互消息、Kira 结果取回和loop/tree DE/scaling; package 仍只验证并导入用户在外部分生成的完整reduction。

A 公开函数汇总

函数	用途	主要option/缺省
DSKinematics	提议或审计结构/routing、双动量列表、动力学变量规则、秩、零空间与可逆性	rules=Automatic (缺省提案)
DSInit	解析family、注册context、生成topology/sector/convention metadata	KinematicRules=Automatic; 写初始化文件=False; 写导数算符=False; 覆盖=False
DSInfo	返回当前初始化摘要与readiness	无
DSPParameterNotation	返回缺省/当前根号notation 与用户变量	context=当前已注册context
DSRedefineParameters	用完整新规则重新初始化参数坐标	进度=Automatic
DSMessagesOn/Off/Q	开关或查询非致命提醒; fatal error 始终保留	无
DSPublicAPI	返回公开函数分组、清单与高层缺省options	无
DSSeeds	生成全可达sector canonical seeds; timeOnly 自动选择direct vertex-pack 或保留massless n 的fixed line-pack	离散态=Automatic (由preset 决定); 生成shrink sector=True; 规模门禁=Automatic
DSLinear	seed 转后端中立linearData	系数规则=Automatic; Kira ordering=Automatic
DSKiraExport	从linearData 写Kira 基础输入与映射	输出目录=None; active basis=None; job options=Automatic
DSKiraImport	读回完整Kira reduction/master 输出	结果/master/log 文件=Automatic; 进度=Automatic
DSDE	用ds 与reduction 构造同序DE	变量=初始化全部; 最大迭代=100; 输出目录=None
DSScaleCheck	检查top/residual Euler scaling	关系=Custom; 变量/权重/次数=Automatic
DSTreeSeeds	生成direct pure-time/tree sector rules; loop 投影只作交叉验证	sector=all; 保留source=True
repIterative	tree 积分迭代到指定 a_e 终点	终点=全零; 最大步数=Automatic
DSTreeNaiveIBP	联立投影后的tree time-IBP 并解一步升幂对象	masters=直接dlog 同序列表; 进度=Automatic
DSTreeNaiveDE	用naive tree IBP 构造同序normalized-master DE	变量=顶点与massive-leg energies; 进度=Automatic
DSTreeDLogDE	生成tree dlog connection、letters 和同序masters	无
dtau	指定顶点的time-IBP	显式topology 或当前context
dqq/dqk	指定loop generator 的momentum-IBP	显式topology 或当前context
ds	对带动力学系数的 J 线性组合求总导数	变量必须是初始化外部名字
rep2innerform	用户/内部标量积表示转换, 不改 J 指标;	显式topology 或当前context
rep2outform	无唯一逆映射时inner 明确失败	
rep2Integrand	把loop 指标积分展开成惰性被积函数	不积分、不生成IBP
symmetry	应用自动tadpole 与用户symmetry rules 的并集	单次应用, 不自动重复
repSymmetry0	返回输入中的原始用户symmetry rules	无

表中的原子、tree 与DS* 高层接口均由016 当前主线提供。各函数的精确可用options 以Options[函数] 为准；未实现接口不得预定义成貌似成功的空壳，也不得静默回退到不完整结果。